PYTHON SUMMIT 2026 · 20 PAŹDZIERNIKA · WARSZAWA

Ruff, uv i... ty

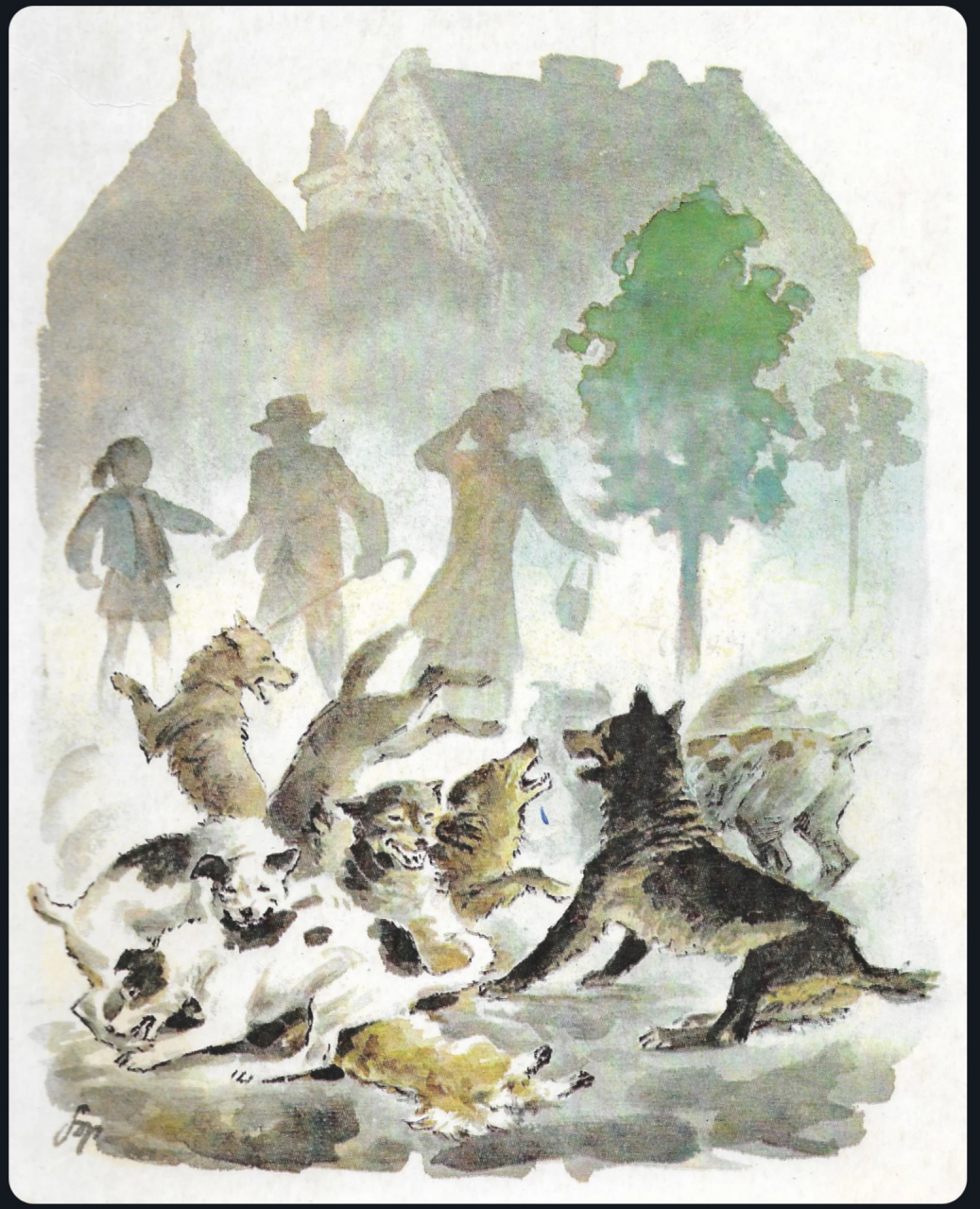
Sekunda zamiast minuty:
toolchain Pythona od Astrala



Marcin Bardź · https://github.com/tupteq/ruff-uv-ty-python-summit-2026

AKT I · PROBLEM

Goście



1 · PROBLEM

15 narzędzi, 15 konfiguracji

pip · pip-tools · venv · virtualenv* · pyenv* · pipx · Poetry* · twine

flake8* · Pylint* · isort · Black · pyupgrade · mypy* · Pyright

DO TEGO PLIKI KONFIGURACJI

`setup.cfg` · `.flake8` · `tox.ini` · `mypy.ini` · `.isort.cfg` · `requirements-dev.txt`

* łącznie około 700† pluginów; ponad 100‡ z nich już wbudowane w Ruff, uv i ty
† z czego około 300 jest aktywnie rozwijanych
‡ w tym 17 z 20 najczęściej pobieranych pluginów flake8

1 · PROBLEM

Ile kosztuje ten bałagan

Czekanie

Przykład: 2 min × 15 uruchomień × 10 osób = 5 h dziennie

Onboarding

Kilka narzędzi do postawienia przed pierwszym commitem

Konflikty

isort kontra Black, inne wersje narzędzi u każdego

„Jeszcze raz”

Wolne sprawdzenie odpalasz rzadziej, więc błędy wychodzą później

Kiedy sprawdzenie trwa sekundę zamiast minuty, robisz je częściej.
Ty i twój agent AI.

1 · PROBLEM

Trzy narzędzia zamiast piętnastu

uv

Pakiety, środowiska i sam Python

ZASTĘPUJE

pip, pip-tools, venv, virtualenv, pyenv, pipx, Poetry, twine

Gotowy na produkcję

Ruff

Lint i formatowanie

ZASTĘPUJE

flake8 z pluginami, isort, Black, pyupgrade, część Pylinta

Gotowy na produkcję

ty

Typy i podpowiedzi w edytorze

ZASTĘPUJE

mypy, Pyright

Beta · wersja 0.0.89

1 · PROBLEM

Astral: kto za tym stoi

2022

Ruff: linter napisany w Ruście

2024

uv: od zamiennika pip do całych projektów

2025

ty: type checker, beta od grudnia

2026

OpenAI ogłasza przejęcie (19 marca)

1 plik

binarny, w Ruście. Nie potrzebuje Pythona, żeby zainstalować Pythona.

Ruffa używają m.in.

PROJEKTY	FIRMY
Transformers	Amazon
LangChain	Microsoft
PyTorch	IBM
FastAPI	Netflix
Home Assistant	Mozilla

~140 mln

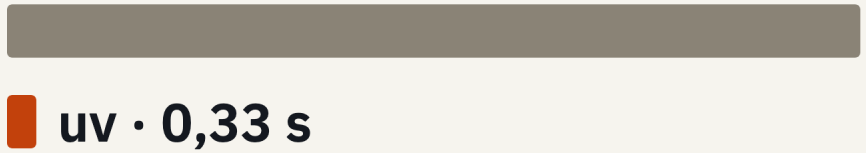
pobrań uv z PyPI w ciągu 30 dni, razem z CI

1 · PROBLEM

O ile szybciej: trzy pomiary

Instalacja zależności

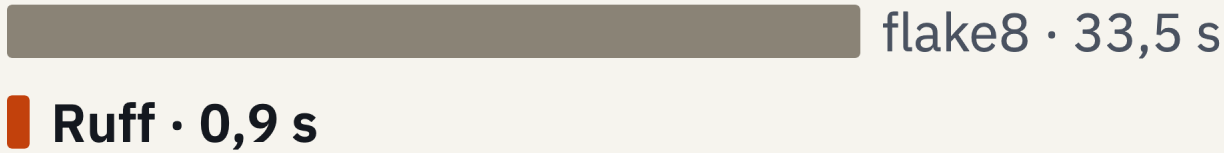
80 pakietów, rozgrzany cache, bez kompilacji `.pyc`



29×

Lint całego repo

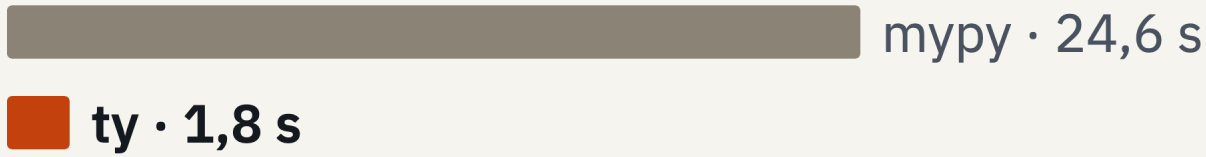
Django, 529 tys. linii; Ruff sprawdza 407 reguł



37×

Sprawdzanie typów

Home Assistant, bez cache; benchmark Astrala



14×

Pomiar 18.10.2026 (Linux, 2 rdzenie): uv 0.13.1, pip 26.2.1, Ruff 0.17.1, flake8 7.4.1 · typy: BENCHMARKS.md w repo ty

AKT II · W PRAKTYCE

Tresura



2 · W PRAKTYCE

uv: jedno narzędzie do całego cyklu

Python	instaluje i przełącza wersje, także 3.15(t) (wydany 9 października)
Środowisko	tworzy `.venv` i sam go pilnuje — bez ręcznej aktywacji
Zależności	zapisuje `uv.lock` — jeden plik dla macOS, Linuksa i Windowsa
Narzędzia	uruchamia je bez instalacji: `uvx ruff`, `uvx ty`
Paczki	buduje i publikuje, zamiast build i twine

29×

szybciej niż pip w naszym pomiarze; Astral deklaruje 10–100×

0.13.1

najnowsza wersja, 14.10.2026

MIT lub Apache 2.0

licencja do wyboru

Status w PyPI

Production/Stable

2 · W PRAKTYCE

Nowy projekt w trzech komendach

PROJEKT

```
$ uv init demo && cd demo
$ uv add httpx
$ uv run demo
```

→ brakujący Python pobiera się sam

→ powstają `pyproject.toml`, `.venv` i `uv.lock`

→ `uv run` sam synchronizuje środowisko

SKRYPT W JEDNYM PLIKU (PEP 723)

```
$ uv init --script skrypt.py
$ uv add --script skrypt.py httpx
$ uv run skrypt.py
```

→ zależności w komentarzu na górze pliku

→ skrypt dostaje własne środowisko

→ jeden plik: dla kolegi albo dla agenta AI

Nowa osoba w zespole: `git clone … && uv sync` i może pracować.

2 · W PRAKTYCE

Ruff: linter i formatter w jednym

Szczeka 954 regułami. Nie gryzie: ryzykowne poprawki tylko na żądanie.

954 reguły (815 stabilnych), m.in. z Pyflakes, isort, pyupgrade i flake8-bugbear

407 reguł włączonych domyślnie w 0.17.1; przed wersją 0.16 (lipiec 2026) było ich 59

mypy też lintuje się Ruffem — tak jak pip, Poetry, PDM i Pylint

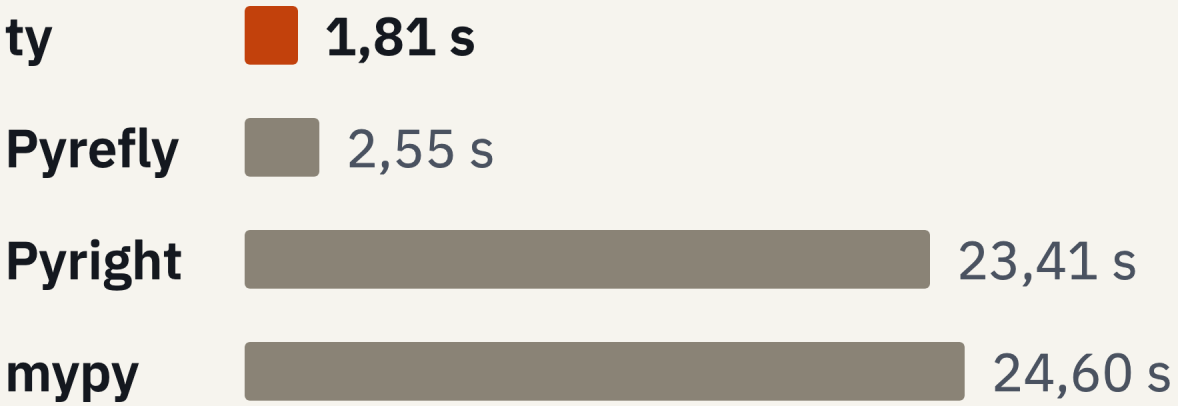
PRAWDZIWY BŁĄD: REGUŁA B015

```
def test_total():
    result = total([1, 2])
    result == 3 # B015
```

Zapomniany **assert**: porównanie nic nie robi, więc test zawsze przechodzi. Ruff wyłapuje to bez instalowania pluginu.

ty: diablo szybki, ale jeszcze beta

HOME ASSISTANT, SPRAWDZENIE BEZ CACHE



STATUS

beta, wersja 0.0.89; stabilna obiecana na 2026

PYDANTIC

wspierany od lipca 2026

DJANGO

jeszcze nie; obecnie Pyrefly od Mety

UŻYWAJĄ

FastAPI, Typer, SQLModel

Źródło: BENCHMARKS.md w repo ty, benchmark Astrala; ty 0.0.55, Pyrefly 1.1.1, Pyright 1.1.410, mypy 2.1.0

Jedna konfiguracja: pyproject.toml

```
[project]              # zależności (PEP 621)
[dependency-groups]    # dev, test (PEP 735)
[tool.uv]              # ustawienia uv
[tool.ruff]            # lint i format
[tool.ty.rules]        # poziomy reguł ty
```

ZNIKA

`setup.cfg`, `.flake8`, `mypy.ini`, `.isort.cfg`, `requirements-dev.txt`

ZOSTAJE OSOBNO

`uv.lock` (generowany), `.python-version`, konfiguracja CI

Jeden plik zamiast pięciu: `pyproject.toml` konfiguruje uv, Ruffa i ty.

Przykładowy pyproject.toml

PROJEKT I UV

```toml
[project]
name = "demo"
version = "0.1.0"
requires-python = "⩾3.12"
dependencies = ["httpx"]

[dependency-groups]
dev = ["ruff==0.17.1", "ty==0.0.89"]

[tool.uv]
# starszy uv odmówi pracy
required-version = "⩾0.13"
```

RUFF I TY

```toml
[tool.ruff.lint]
# jawnie zamiast 407 domyślnych
select = ["E", "F", "I", "UP", "B"]

[tool.ty.rules]
# na start same ostrzeżenia
all = "warn"

[tool.ty.terminal]
# ostrzeżenia nie wywalają CI
error-on-warning = false
```

2 · W PRAKTYCE

To samo sprawdzenie w trzech miejscach

Edytor

Ruff i ty jako language server — błąd widać w trakcie pisania.

PyCharm 2025.3+: uv, Ruff i ty wbudowane.
VS Code: rozszerzenia.

Commit

Hook z Ruffem trwa ułamek sekundy — takiego nikt nie wyłącza.

Uruchamiany przez `uv run`, więc wersja Ruffa pochodzi z `uv.lock`.

CI

```
$ uv sync --locked
$ uv run ruff check
$ uv run ruff format --check
$ uv run ty check
```

`--locked` przerywa build, gdy `uv.lock` nie zgadza się z `pyproject.toml`.

Wszędzie to samo: `uv.lock` pilnuje wersji, `pyproject.toml` — reguł.

AKT III · CO DALEJ

Nowy właściciel



3 · CO DALEJ

Agenci AI i szybkie sprawdzenia

Generuje kod → **Sprawdza: lint, typy, testy** → **Czyta diagnostykę** → **Poprawia**

…i od nowa, wiele razy na jedno zadanie — więc każda sekunda sprawdzenia się mnoży.

Diagnostyka ty: pisana z myślą o ludziach i agentach (ogłoszenie bety, grudzień 2025).

PEP 723 + `uv run`: skrypt agenta dostaje własne środowisko, bez instalacji.

Do AGENTS.md albo CLAUDE.md — po każdej zmianie agent uruchamia:

```
uv run ruff check --fix · uv run ruff format · uv run ty check · uv run pytest
```

„Gdybyśmy mergowali każdy PR od Claude'a, zniszczyłoby to projekt."

Charlie Marsh, Astral — podcast Test Set, luty 2026 (tłum.)

3 · CO DALEJ

OpenAI przejmuje Astral: fakty

19 MARCA 2026

ogłoszenie przejęcia; zespół dołącza do Codex

WIOSNA 2026

finalizacja — oficjalnej daty brak

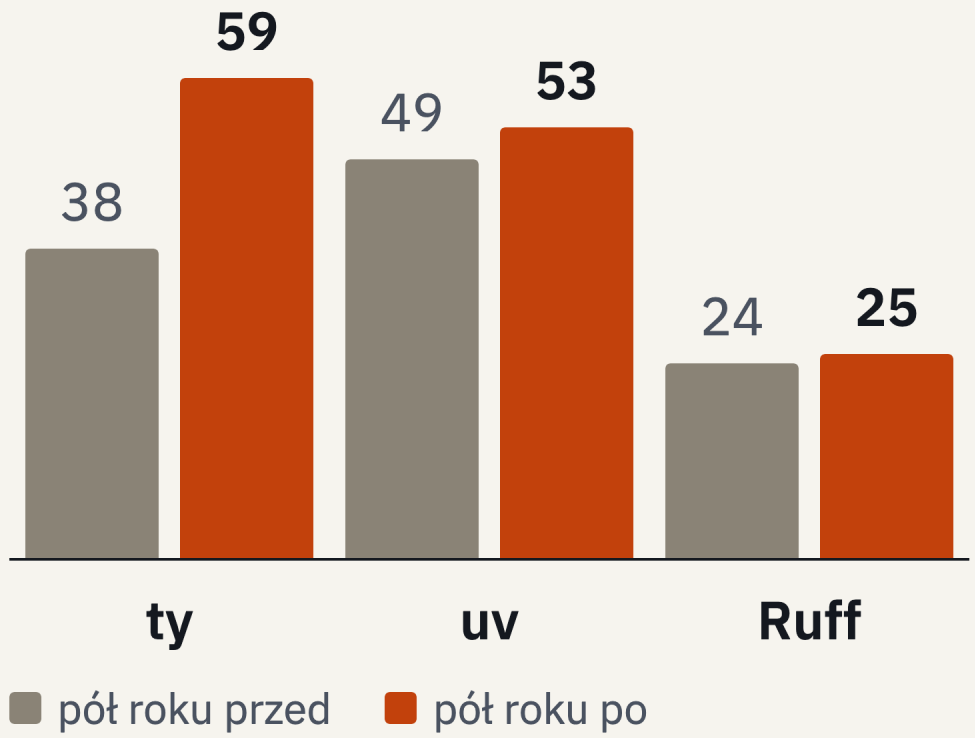
LICENCJE

Ruff i ty: MIT, uv: MIT lub Apache 2.0 — każdy może zrobić fork

PYX

komercyjny rejestr wygaszany; indeks GPU otwarty dla wszystkich

WYDANIA W PYPI: PÓŁ ROKU PRZED I PO 19.03



Źródła: blog Astrala i ogłoszenie OpenAI (19.03.2026) · Talk Python #552 (czerwiec 2026) · PyPI

3 · CO DALEJ

Ryzyko i plan wyjścia

Ryzyko

Priorytety mogą iść za potrzebami OpenAI i Codexa

Trzy kluczowe narzędzia utrzymuje jedna firma

Wersje 0.x mogą przynosić zmiany łamiące

Co robisz już dziś

Przypinasz wersje (krótka smycz), a nowe wpuszczasz z tygodniową karencją:

```
exclude-newer = "7 days"
```

Szesnasty standard? Nie: PEP 621 · 723 · 735 · 751

Wyjście jedną komendą:

```
uv export --format pylock.toml
```

Nie musisz ufać — wystarczy mieć wyjście.

3 · CO DALEJ

Migracja w czterech krokach

1 **CI bez zmian w kodzie.** `uv pip install -r requirements.txt` zamiast pip — zysk tego samego dnia.

2 **Lock.** Zależności do `[project]`, potem `uv lock`, a w CI `uv sync --locked`. Z Poetry lub Pipenv: `uvx migrate-to-uv`.

3 **Ruff.** Zacznij od `select = ["E", "F"]` i dokładaj reguły co sprint. Formatowanie całego repo jednym commitem.

4 **Przejście na ty.** Najpierw jako ostrzeżenia, obok mypy. Przełącz, gdy mypy daje więcej szumu niż pożytku.

Każdy krok daje zysk osobno i każdy da się cofnąć.

3 · CO DALEJ

Pułapki, które warto znać

uv

`uv sync` usuwa z `.venv` wszystko, czego nie ma w `uv.lock`

`uv pip` nie czyta `uv.lock`: osobny tryb zgodny z pip

Ruff

Aktualizacja do 0.16+ bez jawnego `select`: ok. 400 reguł zamiast 59 i czerwone CI

`--unsafe-fixes`, czyli zdjęty kaganiec: potrafi zmienić działanie kodu

`--add-ignore .` gryzie poza repo: tylko przy czystym `git status`

ty

Wersje 0.0.x zmieniają diagnostyki — przypinaj dokładną wersję

Brak systemu pluginów, np. dla Django — tu dziś Pyrefly

3 · CO DALEJ

Wnioski dla praktyków i biznesu

Dla praktyków

- 3 narzędzia zamiast 15 (+100 pluginów)
- Jedno miejsce na konfigurację
- To samo sprawdzenie w edytorze, hooku i CI
- Wynik w (mili)sekundach, nie w minutach

Dla biznesu

- W skali miesiąca oszczędzasz godziny czasu ludzi. Sam rachunek za CI spadnie o 36 USD miesięcznie — tyle co worek karmy*.
- Onboarding: `git clone` i `uv sync`
- Łańcuch dostaw: hashe w `uv.lock`, `uv audit` (preview)
- Tanie wyjście dzięki standardom

* 3 min krócej × 2000 przebiegów miesięcznie × 0,006 USD za minutę (GitHub Actions, Linux; tenki.cloud, 2026)

JUTRO RANO

`uvx ruff check --statistics`

Sprawdź w swoim repo, bez instalacji. Wynik w sekundę.

Lider zespołu: krok 1 migracji to jedno popołudnie.
Zmierz CI przed i po.



Ruff, uv i… teraz ty. Pytania?

Marcin Bardź

github.com/tupteq/ruff-uv-ty-python-summit-2026

Oceń wystąpienie

Prezentacja

Ilustracje: Konstanty Sopoćko do książki Jana Grabowskiego „Puc, Bursztyn i goście” (Nasza Księgarnia)*
* Poza grafiką na tym slajdzie, która pochodzi ze strony https://ladaco.blogspot.com†
† i została wykonana na podstawie zdjęcia z galerii https://www.deviantart.com/malleni-stock/

BONUS · NA PYTANIE Z SALI

uv i Ruff w monorepo

```
# pyproject.toml w korzeniu repo
[tool.uv.workspace]
members = ["packages/*", "services/*"]

[tool.uv.sources]
core = { workspace = true }

$ uv sync --all-packages
$ uv run --package api pytest
```

JEDEN LOCK

`uv.lock` w korzeniu repo;
pakiety korzystają z siebie bez publikowania

WYBÓR PAKIETU

`--package api` uruchamia jeden pakiet,
`--all-packages` instaluje wszystko

RUFF TEŻ

każdy pakiet może nadpisać reguły
we własnym `pyproject.toml`

Źródło: dokumentacja uv (Workspaces)